

scripts/pipeline/phase-07-closure.sh — User Guide

Last verified: 2026-09-18 (feature 002-build-test-distribute-pipeline, task T055) **Inheritance:** HelixConstitution §11.4.18 (script documentation mandate); Lava §6.J (Anti-Bluff), §6.T.3 (no force-push), Decoupled Reusable Architecture rule's Automated Pipeline Pin-Advance Path

Overview

The pipeline's **repository closure** phase (FR-014/FR-015/FR-016/FR-017, tasks.md T055). It is the last phase in a full pipeline run: it advances every submodule this project has authorized the pipeline to advance unattended, then commits and pushes all outstanding main-repository changes — including any updated pins — to every configured upstream.

The T055 entry in specs/002-build-test-distribute-pipeline/tasks.md **is this script's design document** — there is no separate design file. It records three operator decisions (2026-08-26) this script implements exactly:

1. **Recursive scope, per-level invocation.** The submodule set examined is the FULL RECURSIVE set, not just this repository's direct submodules. The script drives the **unchanged** scripts/advance-all-submodules.sh once per level: the main repository, then every already-initialized submodule that itself declares nested submodules. It never advances a level-2+ pin — doing so would require a commit *inside* a level-1 submodule, which is the R-005 step-6 capability that advance-all-submodules.sh removed outright on 2026-08-26 (see that script's own header / docs/scripts/advance-all-submodules.sh.md). A submodule declared but not yet checked out is initialized **at its already-recorded pin** (git submodule update --init, never --remote) so it becomes enumerable and recordable — never so it becomes advanceable; only advance-all-submodules.sh's OWN GOVERNANCE_ALLOW (currently

the single entry helixqa) decides that, unchanged by this phase. constitution is excluded from the walk **entirely**, at every level — never entered, never init'd, never passed as a target repository — per root CLAUDE.md condition (B)'s default-deny on the governance submodule itself.

2. **FR-017 wins.** A run that ends with pins staged but not committed is an intermediate state, never this phase's pass condition. This phase's own advance walk may legitimately leave the tree staged-not-clean; the commit+push+verify step that follows is what reaches FR-017's actual end state.
3. **Declared path set, refuse on surprises.** scripts/commit_all.sh defaults to git add -A. This phase instead computes a *declared* path set — everything dirty when the phase started (which, because FR-000 already guarantees a clean tree at the **start of the pipeline run**, can only be output from this run's own earlier phases) plus whatever this phase's own top-level advance staged — and passes exactly that set to commit_all.sh --files. Anything dirty afterward that is in neither set is a surprise, and the phase refuses to commit it, naming the paths.

Usage

```
scripts/pipeline/phase-07-closure.sh <run_id> [repo-path]
```

<run_id> must already have a report.json (via lib/run-report.sh's init_run_report), exactly like every other wired phase. This script appends one closure phase entry to that report and merges every Submodule Advance Record this run produced — across every level — into its submodule_advances[] array (the merge helper T055's own design note flagged as missing; this script is where it lives).

Test-only environment overrides

Neither is required, or able to relax a safety boundary, for a real invocation — they only make a hermetic fixture reachable, the same posture advance-all-submodules.sh's own --allow-local-path-remotes takes:

Variable	Effect
LAVA_ADVANCE_VERIFY_CMD	Passed straight through to every advance-all-submodules.sh invocation this phase makes. Unset by

Variable	Effect
	default, so production gets that script's own default (a real rebuild-and-test via <code>phase-01-build.sh + phase-02-test.sh</code>).
	When <code>true</code> , adds <code>--allow-local-path-remotes</code> to every <code>advance-all-submodules.sh</code> invocation.
<code>LAVA_CLOSURE_ALLOW_LOCAL_PATH_REMOTES</code>	A hermetic fixture's "upstream" is a local bare repo, which that script otherwise refuses to fetch from by design (root <code>CLAUDE.md</code> condition (C)).

Exit codes

Code	Meaning
0	closure complete: every level examined and recorded, the main repository committed and pushed to every configured upstream (or there was genuinely nothing to commit because the tree was already at the end state), working tree empty, 0 recursive submodule entries showing + or -
1	closure failed: an advance was rejected somewhere this phase treats as a real failure, an undeclared path appeared, the commit/push step failed (including a \$6.C mirror-divergence, which <code>commit_all.sh</code> 's own push step already detects), or the post-closure FR-017 verification found the tree not clean or not fully initialized
2	usage/precondition error (missing <code>run_id</code> , absent <code>report.json</code> , unreadable repo path). Nothing attempted

Anti-bluff properties

1. **Declared-path reconciliation, not `git add -A`.** Verified by `tests/pipeline/test_phase_07_closure_hardening.sh`'s I1 case: a deliberately-injected stray untracked file during the advance step is detected and refused, never silently committed.

2. **FR-017 is checked directly, not inferred from a successful commit.** `git status --porcelain --ignore-submodules=none` and `git submodule status --recursive` are read for real after closure. Falsifiability-rehearsed (hardening suite case I3): neutering the `dirty_recursive` guard makes a run with a genuinely-uninitialized nested submodule wrongly report success; reverting restores the correct FAIL.
3. **Per-submodule evidence at every level, merged exactly once.** The hardening suite's I4 case exercises a 3-submodule, 2-level fixture producing a mix of `ADVANCED` / `REFUSED_GOVERNANCE_DENY` / `NO_NEWER_COMMIT` outcomes and confirms every one lands in `report.json`'s `submodule_advances[]`, and that re-running the merge does not duplicate entries.
4. **Honest scope note, not a false completeness claim.** `scripts/advance-all-submodules.sh` reached its current structurally-safe shape only after five adversarial review rounds (T054, accepted 2026-09-18). This phase has **not** yet been through an equivalent adversarial pass against a real submodule tree — it is a first implementation, exercised only by its own two hermetic test suites so far. `tasks.md` T056 (a disposable-clone dry run) and T062 (a disposable-branch, full end-to-end run) are what exist to close that gap next; this script's own header states this plainly rather than implying a maturity it has not earned yet.

Known defects found and fixed during T055's own test-writing (2026-09-18)

Two independent subagents, writing separate test suites in parallel worktrees, each independently discovered and fixed the same two real bugs in the first draft of this script — recorded here because "found by construction of the tests, before any real invocation" is exactly the anti-bluff discipline this project's testing culture requires:

1. **`commit_all.sh` path resolution.** `scripts/commit_all.sh` derives its own repository root from its own `BASH_SOURCE` and unconditionally `cds` there — it has no `repo-path` override of its own. The first draft resolved `COMMIT_ALL_SCRIPT` relative to *this script's* location rather than the *target* repository, so a `[repo-path]` override never actually redirected the `commit/push` step. Invisible in production (same checkout either way); fatal against any disposable clone or test fixture. Fixed: resolved via `${REPO_ROOT}/scripts/commit_all.sh`.
2. **`LAVA_ADVANCE_VERIFY_CMD` passthrough syntax.** The original `$ {LAVA_ADVANCE_VERIFY_CMD+LAVA_ADVANCE_VERIFY_CMD="$LAVA_ADVANCE`

`_VERIFY_CMD"} idiom is not a valid bash env-var-prefix assignment — bash recognizes NAME=value command prefixes syntactically at parse time, and a word starting with ${ is never reclassified as one regardless of what it expands to. Bash instead tried to execute the expanded string as a command, failing with exit 127 — which the exit-code handling did not distinguish from a normal non-zero outcome, so advance-all-submodules.sh could silently never run at all while the phase still reported success. This was production-reachable (not test-only): any real, non-trivial verify command supplied via this variable would have hit the same failure mode on its very first invocation. Fixed: built into a proper NAME=value array passed to env, with an explicit catch-all for any undocumented advance-all-submodules.sh exit code.`

Maintenance

When this script is modified, update this document in the same commit (§11.4.18 / CM-SCRIPT-DOCS-SYNC convention).

Cross-references

- `scripts/pipeline/phase-07-closure.sh` — the script itself
- `scripts/advance-all-submodules.sh` — the per-level advance engine this phase drives unchanged (`docs/scripts/advance-all-submodules.sh.md` if present; otherwise see that script's own header)
- `scripts/commit_all.sh` — the commit+push wrapper this phase's Step 3 invokes with a declared `--files` set
- `tests/pipeline/test_phase_07_closure.sh` — core hermetic suite (governance allow/deny, per-level recursion, constitution exclusion, uninitialized-submodule init)
- `tests/pipeline/test_phase_07_closure_hardening.sh` — adversarial suite (surprise-path refusal, commit/push failure propagation, FR-017 verification, `submodule_advances[]` merge correctness, usage errors)
- `specs/002-build-test-distribute-pipeline/tasks.md` — T055's own text is this script's design document
- `specs/002-build-test-distribute-pipeline/quickstart.md` — Scenario 4, Step B